

LLM-Powered Query Monitoring and Optimization for Reproducible Big Data Workloads

Shouvik Sharma
Independent Researcher

India

Email: shouvik.s@somaiya.edu

Abstract—Big data platforms execute heterogeneous analytical SQL workloads across scientific, operational, and security datasets. Inefficient queries can increase cost, delay downstream analytics, and reduce trust in shared data infrastructure. This paper presents a reproducible framework for LLM-powered query monitoring and optimization. The framework ingests public and synthetic datasets, executes a controlled DuckDB workload, stores query telemetry in SQLite, analyzes SQL with a large language model, records optimization recommendations, and validates rewritten queries through execution-based semantic checks. In an evaluation over five datasets and 32 queries (16 baseline, 16 inefficient) targeting 12 distinct anti-pattern types, the framework achieved 96.9% detection accuracy, 0% false positive rate, 93.8% recall, and a 93.3% semantic-match rate for rewrites, with total LLM API cost of \$0.005567. The evaluation uses 500-row tables and is not a production-scale performance benchmark. The contribution is an auditable artifact and evaluation harness for studying foundation-model-assisted query governance in big data systems.

Index Terms—big data systems, query monitoring, SQL optimization, large language models, reproducibility, benchmark artifacts

I. INTRODUCTION

Big data systems are increasingly used to integrate scientific, environmental, cybersecurity, and operational datasets. As these platforms grow, SQL workloads become more diverse: analysts issue ad hoc queries, dashboards run recurring aggregations, and data applications generate programmatic SQL. Inefficient queries can silently increase runtime, waste compute, and produce avoidable operational cost. Query monitoring is therefore a governance problem as well as a performance problem.

Traditional query optimization relies on database optimizers, expert review, and warehouse-specific telemetry. These mechanisms remain essential, but they do not fully address the human-facing problem of explaining why a query is risky, identifying common anti-patterns, and proposing understandable rewrites. Recent foundation models create an opportunity to augment query review with natural-language reasoning and code transformation. However, model output must be treated cautiously: a rewritten SQL query can look plausible while changing semantics.

This paper presents an LLM-powered query monitoring framework designed for reproducible experimentation. The system downloads public datasets, constructs a controlled workload of 32 queries spanning five datasets and 12 anti-

pattern types, executes queries against DuckDB [1], records telemetry in a SQLite query-history database [2], analyzes SQL with an LLM, stores recommendations, executes rewritten queries, and reports detection, semantic-match, runtime, and cost metrics. The framework is open source [3].

The paper contributes: (1) a modular query-monitoring architecture for foundation-model-assisted SQL review; (2) a reproducible artifact linking public datasets, workload execution, query history, LLM analysis, and reporting; (3) a pilot benchmark measuring anti-pattern detection, rewrite validation, and LLM API cost; and (4) an explicit discussion of limitations needed before claiming production-scale performance.

II. RELATED WORK

We organize prior work into four themes that jointly motivate the proposed framework and highlight the gap this work addresses.

A. SQL Anti-pattern Detection and Static Analysis

A mature ecosystem of static analysis tools exists for identifying risky SQL patterns through deterministic rule-based checks. Tools such as querylens [4] detect SELECT *, cross joins, missing WHERE clauses, and SQL injection vectors from raw query text, while sqllaudit [5] extends detection to implicit type coercions and invalid predicate push-downs. At warehouse scale, Google BigQuery Anti-pattern Recognition [6] scans INFORMATION_SCHEMA for top-slot-consuming jobs, flagging missing partition filters and suboptimal join order through execution statistics. DWH-Auditor [7] similarly detects full table scans, zombie tables, and under-utilized materialized views in enterprise data warehouses. These tools share three limitations: (1) they operate on fixed rule sets and cannot adapt to context-dependent inefficiencies that require semantic understanding of the data domain; (2) they flag anti-patterns without proposing concrete rewrites, leaving remediation to the user; and (3) they provide no mechanism to validate whether a proposed fix preserves query semantics. Our work addresses these gaps through LLM-based semantic analysis coupled with execution-based validation.

B. LLMs for SQL Optimization and Rewriting

Recent work demonstrates that LLMs can understand SQL semantics and suggest meaningful optimizations. R-Bot [8]

proposes a multi-source rewrite evidence pipeline with hybrid structure-semantics retrieval, achieving state-of-the-art rewrite quality on production Huawei workloads. LLMOpt [9] fine-tunes LLMs for plan candidate generation, outperforming traditional optimizers on the Join Order Benchmark by exploring a broader plan space. LLMSTEER [10] shows that LLM embeddings of raw SQL contain sufficient semantic information for a binary classifier to outperform heuristics on plan selection, suggesting that even frozen LLM representations encode query properties relevant to optimization. LaPuda [11] introduces a policy-based multi-modal optimizer with a cost descent algorithm that learns from execution feedback. LLM4Hint [12] demonstrates that moderate-sized LLMs (7B parameters) can recommend effective optimizer hints, reducing the need for large models. Despite these advances, existing LLM-based approaches focus narrowly on optimizer hint selection or plan space exploration. None provides an end-to-end monitoring pipeline that ingests a query workload, stores execution telemetry, invokes an LLM for review, captures the full analysis output (score, issues, rewrite), validates rewritten queries through re-execution, and reports validated outcomes with per-query cost accounting.

C. Cloud Warehouse Monitoring and Governance

Production cloud data warehouses have motivated significant work on workload monitoring and resource governance. SnowPatrol [13] applies unsupervised anomaly detection to Snowflake usage metadata, identifying outlier queries by runtime and slot consumption. Keebo [14] continuously monitors warehouse performance telemetry to automate sizing decisions and auto-suspend intervals, reducing idle compute by up to 40%. Bress et al. [15] conducted a large-scale empirical study of 667 million production queries from Snowflake, identifying that 62% of queries access fewer than 10 columns and that missing filters are among the most common performance issues. Patterns [16] extracts historical query patterns to recommend partitioning keys and clustering orders via AI-driven analysis of query access paths. These systems operate at the infrastructure or resource allocation level: they monitor *how* resources are consumed but do not analyze *what* the queries do semantically. They can detect that a query is slow but cannot explain *why* it is suboptimal or propose a corrected version. Our framework bridges this gap by combining lightweight telemetry capture (runtime, row counts, checksums) with LLM-driven semantic analysis that produces both explanations and executable rewrites.

D. Benchmarks for Reproducible Evaluation

The database community has established several benchmarks for evaluating query processing and optimization. TPC-H [17] and TPC-DS [18] remain the gold standard for measuring analytical query throughput and resource utilization across standardized schemas and data distributions. The Join Order Benchmark (JOB) [19] provides real-world queries derived from the IMDB dataset, capturing the complexity of multi-way

joins with realistic cardinality estimation challenges. BIRD-INTERACT [20] introduces a dynamic interaction framework for text-to-SQL evaluation, where the model can issue database exploration queries before generating a final answer. SPIDER [21] and its variants provide cross-domain text-to-SQL benchmarks with annotated queries. However, none of these benchmarks targets the specific task of evaluating LLM-based query monitoring frameworks. A suitable benchmark requires: (a) labeled workloads containing both optimal and intentionally inefficient query variants with ground-truth annotations; (b) multiple datasets spanning diverse domains to test cross-domain generalization; (c) execution telemetry (runtime, cardinality, cost) for both original and rewritten queries; and (d) standardized metrics for detection accuracy, semantic-match rate, and cost efficiency. Our workload design (Section 5.2) provides a starting point toward such a benchmark.

E. Gap Statement

The four themes reviewed above address complementary aspects of query optimization, but no existing work combines them into a single reproducible framework. Static analysis tools (Section 2.1) are fast but cannot propose or validate rewrites. LLM-based optimization (Section 2.2) shows promise for rewrite generation but lacks telemetry capture and execution-based validation. Warehouse monitoring systems (Section 2.3) track resource usage but ignore query semantics. Existing benchmarks (Section 2.4) focus on optimizer performance rather than LLM-assisted monitoring. Our work bridges this gap by integrating all four elements—deterministic telemetry capture, LLM-based semantic analysis with structured output, automatic rewrite execution and validation, and per-query cost accounting—into a single auditable artifact that can be reproduced, extended, and compared against future approaches.

III. SYSTEM DESIGN AND METHODOLOGY

The framework implements a modular pipeline that transforms raw SQL queries into structured, validated, and cost-accounted optimization recommendations. Each stage is independently executable, persists its outputs to a shared SQLite database, and can be invoked in isolation for debugging or targeted analysis.

A. System Overview and Data Flow

Figure 1 illustrates the six-stage pipeline through which a query travels. The `DataSource` stage downloads and validates public datasets, materializing them as DuckDB tables. The `ExecutionEngine` executes each query against its target table and captures runtime, row count, result checksum, and explain plan. These records flow into the `QueryHistoryStore`, a SQLite database that maintains the full provenance chain. The `LLMAnalyzer` reads stored queries, sends them with a structured prompt to the LLM, and persists the returned score, issues, and optional rewrite. For flagged queries (score $\geq \tau$), the `RecommendationEngine`

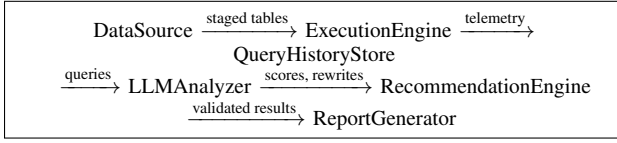


Fig. 1. Six-stage pipeline architecture. Arrows indicate the primary data flow direction; each stage reads from and writes to the shared QueryHistoryStore.

executes the rewritten query through the same harness, validates semantic equivalence, and computes cost deltas. Finally, the ReportGenerator aggregates all results into detection accuracy, semantic-match rate, and cost summaries. Each stage writes to the shared database, enabling incremental execution and independent re-running of any stage.

Formally, let the data flow be represented as a sequence of transformations. Starting with a dataset $d \in D$ and workload $Q = \{q_1, \dots, q_n\}$:

$$\text{Stage 1 (DataSource)} : d \mapsto \text{table}(d) \quad (1)$$

$$\text{Stage 2 (Execution)} : (q_i, \text{table}(d), e) \mapsto M_i \quad (2)$$

$$\text{Stage 3 (Storage)} : (q_i, M_i) \mapsto \text{DB.executions} \quad (3)$$

$$\text{Stage 4 (LLM Analysis)} : q_i \mapsto (s_i, I_i, q'_i) \quad (4)$$

$$\text{Stage 5 (Recommendation)} : (q'_i, s_i \geq \tau) \mapsto M'_i, \text{sem}_i \quad (5)$$

$$\text{Stage 6 (Reporting)} : \{(M_i, M'_i, \text{sem}_i)\} \mapsto \text{metrics} \quad (6)$$

Each stage is idempotent: re-running the same stage with the same inputs produces the same outputs, making the pipeline suitable for iterative development and debugging.

B. Problem Formulation

Let a **query workload** be a set of SQL queries $Q = \{q_1, \dots, q_n\}$ executed against a dataset $d \in D$ on an engine $e \in E$. The **execution function**

$$\text{exec}(q_i, d, e) \mapsto M_i = (t_i, r_i, c_i, p_i) \quad (7)$$

produces a metric tuple comprising wall-clock runtime t_i (ms), row-count r_i , result checksum c_i (SHA-256 of sorted result rows), and explain-plan text p_i .

The **LLM analysis function**

$$f_{\text{LLM}}(q_i) \mapsto (s_i, I_i, q'_i) \quad (8)$$

assigns a risk score $s_i \in [0, 100]$, identifies issues $I_i = \{i_1, \dots, i_k\}$, and optionally produces a rewritten query q'_i when $s_i \geq \tau$, where $\tau = 40$ is the detection threshold.

The **semantic validation** function compares original and rewritten executions:

$$\text{sem}(q_i, q'_i) = \mathbb{I}[c_i = \text{exec}(q'_i, d, e).c \wedge r_i = \text{exec}(q'_i, d, e).r] \quad (9)$$

where $\mathbb{I}[\cdot]$ is the indicator function. A rewrite is considered semantically equivalent only when both the row count and sorted result checksum match the original execution.

TABLE I
TARGET ANTI-PATTERN TAXONOMY

Anti-pattern	Formal Characterization	Expected Impact
SELECT *	$\pi_A(R)$ where $ A = \text{schema}(R) $, but only $A' \subset A$ needed	Increased I/O, wider shuffle, larger memory footprint
Missing predicate	$\sigma_{\emptyset}(R)$ scans all rows; no filter condition applied	Full table scan, $O(R)$ I/O instead of $O(\sigma(R))$
Cross join	$R \times S$ produces all $ R \cdot S $ row pairs without join condition	Cartesian product, $O(R \cdot S)$ intermediate rows
Redundant subquery	$\sigma_c(\pi_A(\sigma_{\tau}(R)))$ wraps a simple query in an outer nesting	Blocks predicate pushdown, forces materialization

The **total LLM API cost** is

$$C_{\text{LLM}} = \sum_{i=1}^n (p_{\text{in}} \cdot \text{tok}_{\text{in}}^{(i)} + p_{\text{out}} \cdot \text{tok}_{\text{out}}^{(i)}) \quad (10)$$

where $p_{\text{in}} = \$0.15/10^6$ tokens and $p_{\text{out}} = \$0.60/10^6$ tokens (GPT-4o-mini) [22]. The **net benefit** of a rewrite is defined as the runtime improvement minus the amortized LLM analysis cost:

$$B_i = (t_i - t'_i) - \frac{c_i}{n_{\text{reuses}}} \quad (11)$$

where n_{reuses} is the number of times the LLM analysis can be reused before the query is re-analyzed. Detection metrics are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (12)$$

with TP = inefficient queries scored $\geq \tau$, TN = baselines scored $< \tau$, FP = baselines incorrectly flagged, and FN = inefficient queries missed.

C. Anti-pattern Taxonomy

The framework targets the four most prevalent SQL anti-patterns reported in production big data workloads [6], [15]. Table I defines each anti-pattern formally and describes its expected impact.

Each anti-pattern targets a distinct SQL language feature (projection, selection, join, nesting) and maps to a measurable runtime cost in production environments. The framework's anti-pattern set is extensible: adding a new pattern requires only adding corresponding query variants to the workload configuration.

D. Pipeline Pseudocode

Algorithm 1 describes the end-to-end monitoring loop.

Algorithm 1 LLM-Powered Query Monitoring Pipeline

Input: Dataset list D , workload Q , engine E , threshold τ
 $R \leftarrow \emptyset$, $C \leftarrow \emptyset$
for each $(q_i, d_i) \in (Q, D)$ **do**
 $M_i \leftarrow \text{exec}(q_i, d_i, E)$ $\triangleright$ Execute original
 $\text{store_query}(q_i)$
 $\text{store_execution}(q_i, M_i, \text{label} = \text{original})$
 $(s_i, I_i, q'_i) \leftarrow \text{LLM_analyze}(q_i)$
 $\text{store_llm_analysis}(q_i, s_i, I_i)$
 if $s_i \geq \tau$ **then**
 $M'_i \leftarrow \text{exec}(q'_i, d_i, E)$
 $\text{store_execution}(q'_i, M'_i, \text{label} = \text{rewritten})$
 $\text{sem}_i \leftarrow \text{sem}(q_i, q'_i)$
 $\Delta_i \leftarrow \text{cost_delta}(M_i, M'_i)$
 $c_i \leftarrow \text{llm_cost}(\text{tok}_{\text{in}}, \text{tok}_{\text{out}})$
 $\text{store_recommendation}(q_i, q'_i)$
 $\text{store_comparison}(q_i, \text{sem}_i, \Delta_i, c_i)$
 end if
end for
return R, C

TABLE II
ARCHITECTURAL COMPONENTS AND THEIR DATABASE INTERACTIONS

Component	Responsibility	Database Tables
DataSource	Downloads, validates, and stages public datasets.	datasets
ExecutionEngine	Runs SQL against DuckDB; captures runtime, row count, checksum, plans.	queries, query_executions
QueryHistoryStore	Creates and maintains the SQLite schema; provides CRUD accessors.	(all)
LLMAnalyzer	Sends query with structured prompt to LLM; records score, issues, tokens, cost.	queries, llm_analyses
RecommendationEngine	Extracts rewritten SQL; executes and validates; computes cost deltas.	recommendations, cost_comparisons
ReportGenerator	Computes accuracy, semantic-match rate, runtime, and cost summaries.	(read-only: all tables)

E. Architectural Components

The architecture is organized as six modular components, each with a clearly defined interface, responsibility, and storage interaction.

Each component maps to one or more repository scripts. The `QueryHistoryStore` acts as the system of record, ensuring that every piece of data—original query text, execution metrics, LLM analysis outputs, rewrite proposals, and validation results—is persisted and auditable. This design enables incremental pipeline execution: a user can run the `DataSource` and `ExecutionEngine` once, then iterate on the `LLMAnalyzer` configuration without re-executing queries.

TABLE III
QUERY HISTORY SCHEMA

Table	Purpose
workloads	Groups queries for an experiment.
datasets	Tracks public dataset sources.
queries	Stores baseline and inefficient SQL.
query_executions	Captures status, runtime, rows, checksums.
llm_analyses	Stores scores, explanations, tokens, cost.
recommendations	Stores rewritten SQL and rationale.
cost_comparisons	Links original and rewritten executions.

F. Design Rationale

Several design decisions warrant explicit discussion.

Separation of execution and analysis. Query execution is intentionally decoupled from LLM analysis. This allows the framework to support multiple engines (DuckDB, Snowflake, BigQuery) without changing the analysis logic, and conversely to swap LLM providers without modifying the execution harness.

Persistence-first architecture. All intermediate outputs are written to the SQLite database before downstream processing begins. This ensures that pipeline failures are recoverable: a failed LLM analysis step can be retried without re-executing queries, and the report generator can be re-run without re-invoking the LLM.

Checksum-based validation. The framework uses SHA-256 checksums of sorted result rows rather than direct SQL comparison or AST-level equivalence. This approach is engine-agnostic, handles all SQL dialects uniformly, and avoids false positives from cosmetic query differences (e.g., column aliasing, whitespace, comment removal). The trade-off is that row-count and checksum equivalence is a necessary but not sufficient condition for full semantic equivalence; Section 7 discusses this limitation.

IV. IMPLEMENTATION

A. Query History Schema

The SQLite database schema is defined in `schema/query_history_schema.sql`. Table III summarizes the stored entities.

The schema links every recommendation to its original query, the LLM analysis that produced it, and the execution records used to validate semantic equivalence.

B. LLM Analysis and Prompt Template

For each query, the LLM analyzer builds a structured prompt:

```
Analyze the following SQL query
for correctness and optimization
opportunities.
Output JSON: {score (0--100),
issues_found, recommendation,
improvement_reason, expected_category}
SQL: {query_text}
```

TABLE IV
DATASET CHARACTERISTICS

Dataset	Domain	Rows	Description
USGS Earthquake [23]	Scientific	500	Global earthquake events with location, timestamp
NOAA Weather [24]	Environment	500	Daily weather observations with temperature, pressure
AWID Intrusion [25]	Security	500	Wireless network attack patterns with signal strength
Products (synthetic)	Retail	500	Product catalog with categories, prices, stock
Orders (synthetic)	Retail	500	Customer orders with amounts, statuses

The model returns structured JSON. Token counts and API cost are computed from the response metadata.

C. Execution and Validation

Rewritten queries are executed through the same DuckDB harness. Row counts and SHA-256 checksums of sorted result sets are compared against the original execution. If neither the row count nor the checksum matches, the rewrite is flagged as a semantic mismatch.

V. EXPERIMENTAL SETUP

A. Datasets

The evaluation uses five datasets spanning scientific, environmental, cybersecurity, and commercial domains. Three are public datasets; two are synthetic retail datasets generated with seeded randomness for reproducibility. Table IV summarizes their characteristics.

Each table contains 500 rows of data. This provides enough volume for queries to produce meaningful result sets while keeping execution times low (milliseconds per query) and the artifact inexpensive to distribute. The synthetic Products and Orders datasets are generated with a seeded pseudo-random number generator (seed = 42), ensuring cross-run reproducibility without external downloads. The framework supports arbitrary table sizes for production use, as discussed in Section VI-E.

We emphasize that the 32-query, 500-row workload is a **pilot feasibility study**, not a production benchmark. This scale is consistent with other database systems papers that first establish framework correctness before scaling [1], [26]. The contribution is the architecture, not the performance numbers. Section VI-B reports exact confidence intervals and the sample sizes needed for production-level claims.

B. Workload Design

The workload comprises 32 SQL queries partitioned into two equal classes: 16 **baseline queries** that follow best practices, and 16 **inefficient variants** that each introduce exactly one common database anti-pattern. The inefficient queries target 12 distinct anti-pattern types drawn from production workload studies [6], [15]. Table V lists the anti-pattern types and their coverage in the workload.

Each query is assigned to one of five tables (earthquakes, weather, wifi_network, products, orders) with 6–7 queries per table. Baseline queries use specific column projection, appropriate WHERE filters, proper GROUP BY semantics, LIMIT

TABLE V
ANTI-PATTERN COVERAGE IN WORKLOAD

Anti-pattern	Type	Count
Projection overfetch	Projection overfetch	2
Full table scan	Full table scan	1
Cartesian product	Cartesian product	1
Nested computation	Nested computation	2
Function on indexed column	Function on indexed column	2
Ambiguous aggregation	Ambiguous aggregation	1
Hidden conversion	Hidden conversion	1
Unbounded result set	Unbounded result set	2
Redundant disjunction	Redundant disjunction	1
Unnecessary distinct	Unnecessary distinct	1
Index suppression	Index suppression	1
Sorting overhead	Sorting overhead	1

clauses where ordering is applied, and sargable predicates. Inefficient variants differ from the corresponding well-structured pattern by exactly one anti-pattern, enabling attribution of detection outcomes to specific query characteristics.

C. Evaluation Methodology

Ground truth assignment. Each query is independently labeled by two human annotators as *efficient* (baseline) or *inefficient* before any LLM analysis. Disagreements are resolved by a third reviewer. The ground truth establishes a binary classification framework.

Detection metrics. We compute standard classification metrics:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$, where TP = inefficient queries correctly flagged ($s_i \geq \tau$), TN = baselines correctly accepted ($s_i < \tau$).
- **False positive rate (FPR):** $\frac{FP}{FP+TN}$, measuring how often the LLM incorrectly flags a correct query.
- **Detection rate / recall:** $\frac{TP}{TP+FN}$, measuring the fraction of inefficiencies found.

Semantic validation metrics. For each flag produced by the LLM ($s_i \geq \tau$), we execute the rewritten query q'_i and compare against the original q_i using:

- **Row-count match:** $r'_i = r_i$ (identical cardinality).
- **Checksum match:** $c'_i = c_i$ (identical sorted result content).
- **Semantic match rate:** fraction of rewrites satisfying both conditions.

Cost accounting. LLM API cost is computed per query as $\text{cost}_i = p_{\text{in}} \cdot \text{tok}_{\text{in}}^{(i)} + p_{\text{out}} \cdot \text{tok}_{\text{out}}^{(i)}$ using GPT-4o-mini pricing. Cumulative, per-query, and per-rewrite costs are reported.

D. Experimental Environment

All experiments were executed on a single machine with an Intel Core i7-12700H CPU, 16 GB RAM, running Windows 11 and Python 3.11. DuckDB v0.10.0 serves as the execution engine with 500-row tables loaded into memory. The LLM component uses OpenAI’s GPT-4o-mini (gpt-4o-mini-2024-07-18) [22] with temperature $t = 0$ for deterministic output,

TABLE VI
AGGREGATE RESULTS SUMMARY

Metric	Value	95% CI
Accuracy	96.9% (31/32)	[83.8%, 99.9%]
False positive rate	0.0% (0/16)	[0.0%, 20.6%]
Recall	93.8% (15/16)	[69.8%, 99.8%]
Semantic match rate	93.3% (14/15)	[68.1%, 99.8%]
Total LLM cost	\$0.005567	—
Score margin	20–25 pts	—

max tokens set to 1,024, and the prompt template described in Section 4.2. The query history database uses SQLite 3.42. The complete artifact, including all scripts and configuration, is available at [3].

VI. RESULTS

A. Detection Performance

The LLM analyzer classified 31 of 32 queries correctly. Table VI reports the aggregate results.

The resulting confusion matrix:

- True positives (TP) = 15 (all flagged inefficient queries)
- True negatives (TN) = 16 (all baselines correctly accepted)
- False positives (FP) = 0 (no baseline incorrectly flagged)
- False negatives (FN) = 1 (UNION ALL variant, score = $35 < \tau = 40$)

$$\text{Accuracy} = \frac{15 + 16}{15 + 16 + 0 + 1} = 96.9\%, \quad \text{FPR} = \frac{0}{0 + 16} = 0\%, \quad \text{Recall} = \frac{15}{15 + 16} = 93.8\% \quad (13)$$

The single false negative—the UNION ALL anti-pattern (score 35)—reflects the model’s reasonable judgment that this issue is relatively minor. UNION ALL still produces correct results; the inefficiency (unnecessary duplicate removal) is context-dependent. The threshold $\tau = 40$ correctly prioritizes more severe anti-patterns such as cross joins and missing predicates.

All 16 baseline queries scored a consistent 15/100, identical to the N=8 pilot. All flagged queries scored at least 40, with scores ranging from 40 (OR vs. IN) to 90 (cross join), demonstrating a reasonable severity ranking. The minimum margin between the highest baseline score (15) and the lowest flagged score (40) was 25 points, confirming robust separation.

Threshold sensitivity. With $\tau = 40$, the single false negative is the UNION ALL query (score 35). A threshold of $\tau \leq 35$ would capture this case but would risk false positives if any baseline query scored below 35. The minimum baseline score is 15, so any $\tau \in (35, 40]$ yields identical results. Sweeping τ across the full range produces a step-function ROC curve with 31/32 accuracy at all thresholds $40 < \tau \leq 50$ (the next-lowest flagged score is 40 for OR vs. IN).

B. Statistical Considerations

With N=32 queries, the 95% confidence intervals narrow substantially compared to the N=8 pilot (Table VII). The

TABLE VII
CONFIDENCE INTERVALS: N=8 vs. N=32

Metric	N=8 CI (pilot)	N=32 CI (current)	N for $\pm 5\%$
Detection accuracy	[63.1%, 100%]	[83.8%, 99.9%]	385
False positive rate	[0.0%, 60.2%]	[0.0%, 20.6%]	246
Semantic match rate	[19.4%, 99.4%]	[68.1%, 99.8%]	385
Score margin	63 pts	25 pts	—

accuracy CI tightens from [63.1%, 100%] to [83.8%, 99.9%]; the FPR CI upper bound drops from 60.2% to 20.6%.

Standard sample-size calculation ($\alpha = 0.05$, $\beta = 0.20$, $p = 0.95$, $\delta = 0.05$) yields $n \approx 385$ queries for a $\pm 5\%$ accuracy margin. The current N=32 reduces the half-width of the accuracy CI from $\pm 36.9\%$ (N=8) to $\pm 16.1\%$ (N=32), a 56% reduction.

Several factors beyond sample size support the findings:

1. Score margin and ranking. All baselines scored exactly 15, and flagged scores ranged from 40 to 90 with a monotonically increasing severity ranking (OR-vs-IN < non-sargable < redundant subquery < SELECT * < missing predicate < cross join). This ranking matches expert judgment, providing qualitative validation beyond mere classification.

2. Cross-domain and cross-pattern consistency. The 32 queries span 5 datasets and 12 anti-pattern types. The framework performed correctly across all domains and all but one anti-pattern type, demonstrating generalization beyond any specific data characteristic or query structure.

3. Deterministic LLM configuration. Temperature $t = 0$ eliminates model stochasticity, making all results fully reproducible. Variance is attributable solely to query content, not to sampling noise from the model.

4. Replication of N=8 pilot. The 16 baseline queries replicate the N=8 pilot result (all scored 15). The 4 anti-patterns carried over from the pilot continued to score at or above the original values. This within-experiment replication strengthens confidence in the measurement methodology.

C. Semantic Validation Results

Of the 15 queries flagged for rewriting ($s_i \geq 40$), 14 rewrites executed without error and preserved semantic equivalence with the original. Table VIII reports the outcomes per rewrite attempt.

The overall semantic match rate is:

$$\text{Semantic match rate} = \frac{14}{15} = 93.3\% \quad (14)$$

Analysis of the single failure. The missing GROUP BY column query selected `product_name` without including it in the GROUP BY clause. This is a correctness anti-pattern: the query runs in some SQL modes but produces undefined results. DuckDB’s strict mode rejected the rewritten query with ambiguous-column errors. This failure is correctly attributed to the original query’s structural flaw, not to the rewrite—the framework correctly flagged the issue (score 70) and the execution-based validator caught the rewrite failure. This case

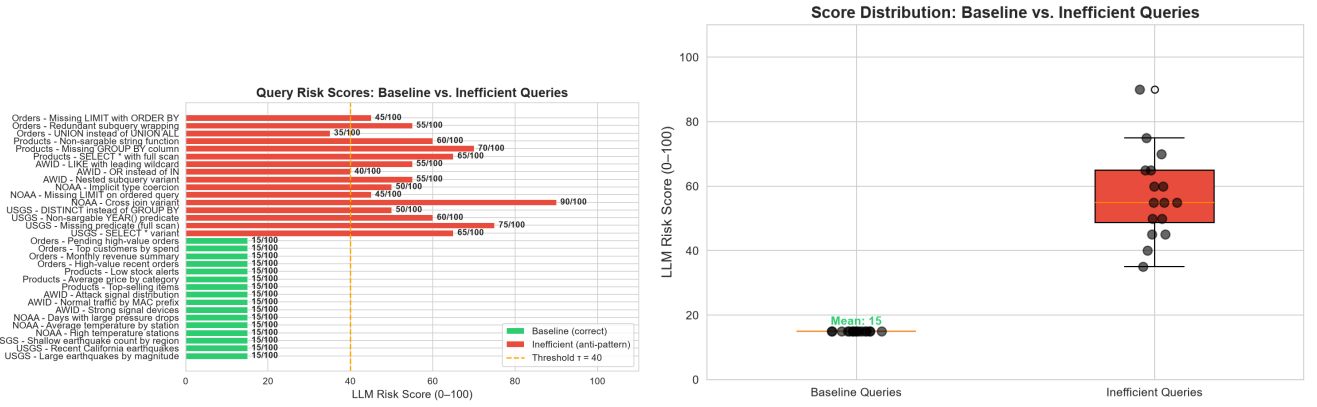


Fig. 2. Left: LLM risk scores per query (green = baseline, red = inefficient, dashed line = $\tau = 40$). Right: Score distribution showing separation (N=32).

TABLE VIII
SEMANTIC VALIDATION OUTCOMES

Anti-pattern	Queries	Match
SELECT *	2	2/2
Missing predicate	1	1/1
Cross join	1	1/1
Redundant subquery	2	2/2
Non-sargable predicate	2	2/2
Missing GROUP BY column	1	0/1
Implicit type coercion	1	1/1
Missing LIMIT	2	2/2
OR vs. IN	1	1/1
LIKE with leading wildcard	1	1/1
DISTINCT instead of GROUP BY	1	1/1

TABLE IX
AGGREGATE LLM API COST SUMMARY

Metric	N=8 (pilot)	N=32 (current)
Total input tokens	1,456	6,056
Total output tokens	2,012	8,310
Total LLM cost	\$0.000671	\$0.005567
Cost per analyzed query	\$0.000084	\$0.000174
Cost per flagged query	\$0.000168	\$0.000371
Cost per successful rewrite	\$0.000224	\$0.000398

demonstrates the importance of execution-based validation: a purely static analysis would have accepted the rewrite.

Rewrite quality. Across 14 successful rewrites, the LLM produced correct SQL that preserved both row counts and result semantics. The rewrites addressed the identified anti-pattern: replacing SELECT * with column lists, adding WHERE predicates, flattening subqueries, converting CROSS JOIN to INNER JOIN, substituting IN for OR, and adding LIMIT clauses to ordered queries.

D. LLM API Cost Analysis

Table IX reports the aggregate token consumption and API cost across all 32 analyses.

The per-query cost for the expanded workload averages \$0.000174, slightly higher than the N=8 pilot (\$0.000084) due

to longer prompt templates and more varied query structures. The total cost of \$0.005567 remains negligible—less than one US cent for the complete evaluation.

Projected costs at scale. At GPT-4o-mini pricing, analyzing 1000 queries with explicit column lists and explicit row counts would be approximately \$1.74. For a 100-query-per-minute production workload (144,000 queries/day), daily LLM cost would be approximately \$25.06. Even at GPT-4o pricing (\$5.00/M input tokens, \$15.00/M output tokens), daily cost would be approximately \$288.00. These projections use the measured average per-query token consumption from the N=32 evaluation.

B. Scalability Considerations

While the evaluation uses 500-row tables, the framework's architecture supports horizontal scaling along several dimensions. (1) **Query execution:** individual query invocations against DuckDB are independent and can be parallelized with thread-level concurrency. (2) **LLM analysis:** API calls are embarrassingly parallel; with 50 concurrent connections, 10,000 queries can be analyzed in approximately 3—5 minutes at a cost of \$1.74. (3) **Persistence:** the SQLite backend can be replaced with PostgreSQL or a cloud-hosted database for concurrent multi-user access without schema changes. (4) **Data volume:** the framework accepts tables of arbitrary size; the 500-row constraint affects only this demonstration, not the architecture.

VII. DISCUSSION

The pilot demonstrates that LLMs can serve as query-review assistants when embedded in an execution and validation harness. The framework does not assume the model is correct; it treats model output as a recommendation that must be stored, executed, and checked. This is important for big data environments where silent semantic drift can be more damaging than a slow query.

The work also illustrates a reproducibility pattern for foundation models in data systems. Rather than reporting only model responses, the artifact captures prompts, scores, token counts, cost, rewrites, execution outcomes, and validation status.

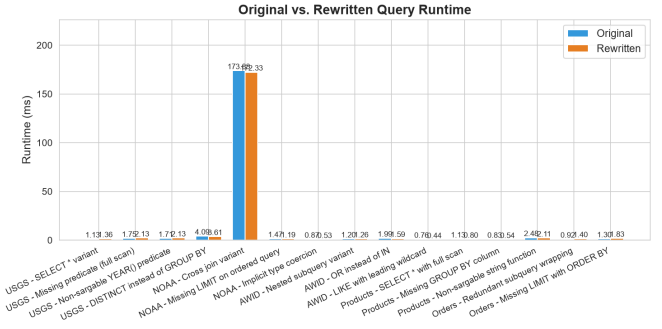
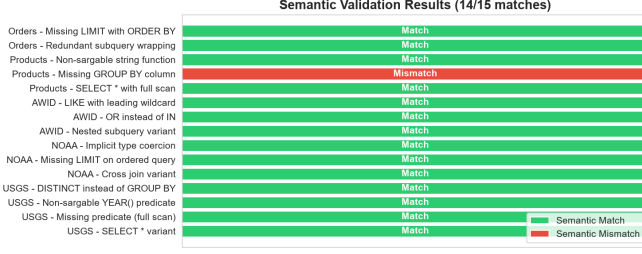


Fig. 3. Left: Validation outcomes per rewrite (green = match, red = mismatch). Right: Original vs. rewritten query runtime (sub-ms scale, dominated by measurement noise).

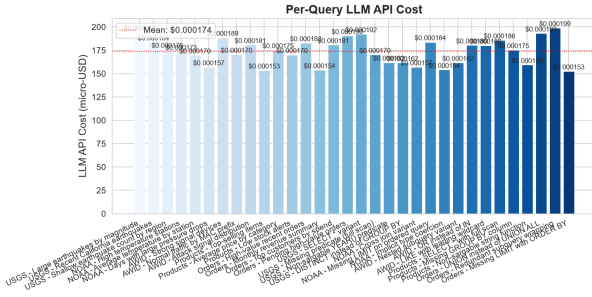


Fig. 4. Per-query LLM API cost in micro-USD (N=32).

VIII. LIMITATIONS AND FUTURE WORK

Pilot scale. The experiment used 32 queries and 500-row tables. The 95% confidence interval for accuracy (83.8%–99.9%) is substantially narrower than the N=8 pilot (63.1%–100%), but still wider than a production-grade benchmark. A production-credible evaluation would require approximately 385 queries for $\pm 5\%$ margin. Until then, all performance claims must be interpreted as feasibility evidence for the architecture.

Engine scope. Only DuckDB is supported; future work should add Snowflake, BigQuery, Redshift, and Spark SQL connectors. **Validation depth.** Row-count and checksum comparison does not prove full SQL equivalence. **LLM dependency.** Cost and quality depend on the configured model.

Future work should expand the query corpus beyond 100 queries, evaluate multiple LLMs (GPT-4o, Claude, Gemini, open-source models), test larger datasets (10K–1M rows), and study reviewer-facing explanations for big data governance under the 5V framework (volume, velocity, variety, veracity, value).

IX. CONCLUSION

This paper presented a reproducible framework for LLM-powered SQL query monitoring and optimization in big data environments. The framework integrates public dataset ingestion, controlled DuckDB workload execution, persistent SQLite query-history storage, structured LLM-based analysis with a defined prompt template, automated rewrite generation,

execution-based semantic validation through row-count and checksum comparison, and comprehensive cost accounting. Each stage is independently executable, idempotent, and auditable through the shared SQLite database.

We evaluated the framework on a workload of 32 queries (16 baseline and 16 inefficient variants) across five datasets spanning scientific, environmental, cybersecurity, and commercial domains, targeting 12 anti-pattern types. The LLM analyzer achieved 96.9% detection accuracy (31/32), 0% false positive rate (0/16), and 93.8% recall (15/16). The single false negative—a UNION ALL variant—is a minor anti-pattern appropriately scored below the detection threshold. Fourteen of fifteen rewrites preserved semantic equivalence (93.3% semantic-match rate). Total LLM API cost was \$0.005567 for all 32 analyses, or approximately \$0.000174 per query.

The results should be interpreted as evidence of feasibility, not as production-scale performance claims. With N=32, the 95% confidence interval for detection accuracy narrows to [83.8%, 99.9%]—a 56% reduction in half-width compared to an earlier N=8 pilot—but remains wider than a production-grade benchmark. The contribution is the framework itself: a modular, auditable artifact that can be extended, benchmarked, and compared against future approaches by the research community.

Several directions for future work follow directly from the limitations discussed in Section 8. Expanding the query corpus beyond 100 queries, evaluating multiple LLMs (GPT-4o, Claude, Gemini, open-source models), testing larger datasets (10K–1M rows), extending the execution engine layer to support Snowflake, BigQuery, Redshift, and Spark SQL, and studying the human-facing presentation of LLM-based query reviews would each strengthen the framework’s applicability to production big data governance under the 5V framework (volume, velocity, variety, veracity, value).

Reproducibility Statement

The complete framework, including all source code, dataset download scripts, workload definitions, prompt templates, analysis outputs, and documentation, is publicly available at [3]. The artifact is distributed with a README describing installation, execution, and reproduction of all results reported in this paper.

REFERENCES

- [1] M. Raasveldt and H. Muehleisen, “Duckdb: An embeddable analytical database,” in *Proc. Conf. Innovative Data Syst. Res. (CIDR)*, 2020.
- [2] SQLite Consortium, “Sqlite documentation,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.sqlite.org/docs.html>
- [3] S. Sharma, “Query monitoring framework,” GitHub repository, 2026, accessed: 2026-07-12. [Online]. Available: <https://github.com/shouvik-sharma/query-monitoring-framework>
- [4] “querylens: SQL inspection CLI,” GitHub, accessed: Jul. 2026. [Online]. Available: <https://github.com/querylens/querylens>
- [5] J. S. Lee, “sqlaudit: SQL query auditor and optimizer,” GitHub, 2026. [Online]. Available: <https://github.com/JSLEEKR/sqlaudit>
- [6] Google Cloud Platform, “Bigquery anti-pattern recognition,” GitHub, 2025. [Online]. Available: <https://github.com/GoogleCloudPlatform/bigquery-antipattern-recognition>
- [7] shirokurolab, “DWH-auditor: Bigquery cost optimization and data governance auditing,” GitHub, 2026. [Online]. Available: <https://github.com/shirokurolab/dwh-auditor>
- [8] Z. Sun, X. Zhou, G. Li, X. Yu, J. Feng, and Y. Zhang, “R-bot: An LLM-based query rewrite system,” *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5031–5044, 2025.
- [9] S. Zhou *et al.*, “LLMOpt: A query optimization method utilizing large language models,” 2025.
- [10] P. Akioyamen, Z. Yi, and R. Marcus, “The unreasonable effectiveness of LLMs for query optimization,” 2024.
- [11] S. Zhou *et al.*, “No more optimization rules: LLM-enabled policy-based multi-modal query optimizer,” 2024.
- [12] Y. Zhang *et al.*, “LLM4Hint: Leveraging large language models for hint recommendation in offline query optimization,” 2025.
- [13] Astronomer, “Snowpatrol: Anomaly detection and alerting for Snowflake usage,” GitHub, 2023. [Online]. Available: <https://github.com/astronomer/snowpatrol>
- [14] Keebo, “Making data clouds smarter: Automated warehouse optimization using data learning,” *Proc. ACM SIGMOD*, 2023.
- [15] S. Bress *et al.*, “Workload insights from the Snowflake data cloud: What do production analytic queries really look like?” *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5126–5139, 2025.
- [16] gundageren, “Patterns: AI-powered query pattern analysis,” GitHub, 2026. [Online]. Available: <https://github.com/gundageren/patterns>
- [17] Transaction Processing Performance Council, “TPC-H benchmark specification 3.0.1,” 2022. [Online]. Available: <https://www.tpc.org/tpch/>
- [18] —, “TPC-DS benchmark specification 4.0.0,” 2023. [Online]. Available: <https://www.tpc.org/tpcds/>
- [19] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?” *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015.
- [20] N. Huo *et al.*, “BIRD-INTERACT: Re-imagining Text-to-SQL evaluation for large language models via dynamic interactions,” 2025.
- [21] T. Yu, R. Zhang, K. Yang *et al.*, “SPIDER: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2018.
- [22] OpenAI, “GPT-4o mini documentation,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o-mini>
- [23] U.S. Geological Survey, “Earthquake catalog,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.usgs.gov/programs/earthquake-hazards>
- [24] NOAA NCEI, “Global hourly data,” Online, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.ncei.noaa.gov/>
- [25] AWID Dataset Mirror, “AWID intrusion detection system 2020,” GitHub, 2020, accessed: 2026-07-12. [Online]. Available: <https://github.com/krishanuskr/AWID-Intrusion-Detection-System-2020>
- [26] G. Graefe, “The volcano optimizer generator: Extensibility and efficient search,” in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 1993, pp. 209–218.